



Università
di Brescia

From Observation to Symbolic Logic: *Learning PDDL Action Models from Observed Plans and Transitions*

Sara Moglia, Luca Puggnetti

Artificial Intelligence

Abstract

This report describes an agent capable of automatically inferring a domain model in PDDL format starting from the observation of executions in a stochastic grid world environment. The agent does not receive the symbolic domain as input, but reconstructs it by observing state transitions—pairs (s, s') produced by successful and failed actions—and applying three sequential mechanisms: a statistical estimation of preconditions and effects conditioned only on successes, an inertia completion that recovers information lost due to partial observability, and a parametric lifting that generalizes concrete predicates into reusable operators with variables. The document is organized into six sections: introduction and problem motivation, state of the art and positioning of the framework, architecture and system characteristics, implementation details, experimental results on scenarios with variable noise and observability, and conclusions.

Contents

1	Introduction	3
2	State of the Art	4
2.1	OLAM	4
2.2	OffLAMPT	5
2.3	PAL	5
2.4	Positioning of our framework	6

3	Agent Characteristics	7
3.1	Environment and symbolic representation	7
3.2	Partial observability	8
3.3	Environment noise (non-determinism)	8
3.4	Hybrid learning: random exploration and manual plans	8
3.5	Final output of the agent	9
4	Agent Implementation	9
4.1	Fundamental mathematical notation	10
4.2	Environment Simulator, <code>GridWorldStochastic</code>	10
4.3	Transition memory and statistical estimation, <code>MiniActionModelLearner</code> .	11
4.4	Inertia Completion, <code>apply_inertia</code>	12
4.5	Parametric inference of operators	13
4.6	Generation of experiences	14
4.7	Validation of inferred operators: two distinct levels	14
4.7.1	<code>check_pddl_inference</code> : structural completeness check	14
4.7.2	<code>is_strictly_valid</code> : UPF-based semantic validator	14
4.8	Iterative experiments and final dashboard	15
4.9	Agent Pseudocode	16
5	Results Obtained	18
5.1	Results of the main scenarios	18
5.2	Qualitative comment on inferred operators	19
5.3	Final graph analysis	19
5.4	Overall Evaluation	22
6	Conclusions	23

1 Introduction

The goal of the project is to build an agent capable of *inferring a PDDL domain starting from observed plans and transitions*, instead of assuming the domain is known a priori. In a classic symbolic planning problem, one starts from a set of already defined operators, with manually written preconditions and effects. In this work, the flow is reversed: the agent observes executions of actions in an environment, records initial and final states, distinguishes successes from failures, and tries to reconstruct a symbolic description compatible with what has been seen.

The motivation for this work arises from a well-documented practical difficulty in the automated planning literature: the manual specification of a PDDL domain requires the designer to explicitly enumerate all the preconditions and effects of every action. In even moderately complex domains, this operation is prone to systematic omissions: a forgotten precondition produces inapplicable plans; a missing effect produces inconsistent states after execution. Replacing this manual specification with an automatic inference process reduces the cost of domain formalization and, above all, allows operating in initially unknown environments where a priori specification is not possible.

In the analyzed project, the problem is made concrete in a *grid world* with a robot and an object to be manipulated. There are three available actions: moving the robot, picking up the object, and dropping the object. The agent must not limit itself to estimating whether a single action succeeds or fails, but must produce a generalized symbolic representation, i.e., a reusable parametric operator on new states. In other words, the system must transition from specific episodes of the type “the robot was in (x, y) and after `pick(box)` obtained certain predicates” to abstract rules of the type “`pick` requires the robot and the object to be co-located and produces `holding`”.

A second objective of the project is to verify how robust this inference process remains in the presence of noise. The project not only studies the ideal case but also models stochastic actuator failures and partial sensor blindness. This aspect is important because a purely symbolic learning agent tends to degrade rapidly when observations are incomplete or when the dynamics are non-deterministic. Therefore, the project aims not only at logical correctness in the perfect case but also at the resilience of the method under more realistic conditions.

Finally, the project has a methodological objective: to build and evaluate an entire domain learning pipeline, from the raw collection of transitions to the production of interpretable parametric PDDL operators. Each phase of this pipeline is described in detail in Section 3 and corresponds to a specific design choice, motivated by the characteristics of the environment (stochasticity, partial observability, rarity of certain actions).

Before describing the system architecture, the next section situates this approach relative to the existing literature on action model learning, identifying the aspects inherited from previous works and the original choices of the proposed framework.

2 State of the Art

Before describing the proposed framework, it is useful to situate it with respect to the three reference works on which it is directly based: OLAM [1], OffLAMPT [3], and PAL [2]. These three systems tackle the problem of learning action models from complementary perspectives, differing in the type of available observations (complete vs. partial, symbolic vs. continuous) and the learning modality (online vs. offline). Their study is the direct starting point for understanding the design choices of the implemented framework.

2.1 OLAM

OLAM (Online Learning of Action Models) [1] is an algorithm for the online and incremental learning of PDDL action models. Unlike offline approaches, OLAM does not require a pre-existing set of traces: the agent actively builds and selects informative traces during execution, directing exploration towards states from which it is possible to extract useful information about the preconditions and effects of the operators.

Formally, OLAM assumes that the predicates P , the names of the operators O , and the constants C are known to the agent, while the environment's dynamics are described by an unknown Ground Truth Model (GTM). In this context, a *lifted* (parameterized) atom is an atom in which concrete constants are replaced by variables: for example, $\text{at}(\text{?robot}, \text{?x}, \text{?y})$ is the lifted version of the concrete atom $\text{at}(\text{robot}, 2, 3)$. Given an operator op with parameters $\text{par}(op)$, $\mathcal{P}(\text{par}(op))$ denotes the set of all lifted atoms that can be formed by applying the predicates in $\mathcal{P}$ to the variables in $\text{par}(op)$. For a move operator with parameters $(\text{?robot}, \text{?from_x}, \text{?from_y}, \text{?to_x}, \text{?to_y})$, $\mathcal{P}(\text{par}(\text{move}))$ includes atoms of the type $\text{at}(\text{?robot}, \text{?from_x}, \text{?from_y})$, $\text{at}(\text{?robot}, \text{?to_x}, \text{?to_y})$, $\text{holding}(\text{?robot}, \text{?from_x})$, and so on. Initializing $\text{pre}(op)$ to $\mathcal{P}(\text{par}(op))$ means starting from the most conservative hypothesis possible: any predicate could be a precondition, and subsequent observations progressively eliminate the unnecessary ones.

The algorithm incrementally maintains five structures for each operator: the set of candidate preconditions $\text{pre}(op)$ (initialized to the complete set of lifted atoms $\mathcal{P}(\text{par}(op))$), the certain positive and negative effects $\text{eff!} + (op)$ and $\text{eff!} - (op)$ (initially empty), the potential positive and negative effects $\text{eff?} + (op)$ and $\text{eff?} - (op)$ (initialized to $\mathcal{P}(\text{par}(op))$), and the set $\text{pre}\perp(op)$ of precondition subsets falsified by failed executions. At each iteration, the agent uses the model learned up to that point to generate a plan towards a state s' from which the execution of an action $op'(c')$ can provide new information: reducing $\text{pre}(op')$ if the action succeeds with false preconditions, adding certain effects if the state changes, or removing potential effects if the state does not change.

OLAM outputs two domains: M , which correctly and fully approximates the GTM from the final state of the algorithm, and $M^{-?}$, which always correctly approximates the GTM. The algorithm terminates in a finite number of iterations. An explicit limitation of OLAM is that it requires full state observability: the algorithm is not designed to handle imperfect sensors or partial observations.

2.2 OffLAMPT

OffLAMPT (Offline Learning of Action Models from Partial Traces) [3] is an offline algorithm for learning action models from sets of partially observable traces, i.e., execution sequences where state observation is incomplete (some truth values of predicates are unknown) and/or actions have missing parameters.

The central problem that OffLAMPT solves is the non-uniqueness of the model compatible with partial traces: two different models could complete the same trace in different ways, both resulting consistent with the observations. OffLAMPT tackles this ambiguity by defining a partial order on action models and searching for minimal models, i.e., the “smallest” models consistent with the traces, in the sense that the enabled transitions and effects produced by M_1 are a subset of those of M_2 . From the set of minimal models, the *cautious model* is then derived, collecting the effects present in all minimal models and the preconditions present in at least one.

The algorithm operates on a 4-tuple structure $(pre?, eff!, eff?, \hat{T})$ where $pre?$ is a superset of true preconditions, $eff!$ is a subset of certain effects, $eff?$ is the set of still potential effects, and $\hat{T}$ is the set of partial traces. Three families of completion rules are applied iteratively: type (1) rules update $pre?$, $eff!$ and $eff?$ by observing literals in transitions; type (2) rules propagate known information about effects into the traces, specifically implementing the principle of inertia (rule 8): if a literal l is present in state $\hat{s}$ before an action $op(c)$ and l is not a potential effect of op , then l is propagated to the subsequent state $\hat{s}'$, since it cannot have been modified by the action; type (3) rules handle cases where the truth value of an unobserved atom is ambiguous, branching the search into two alternatives. OffLAMPT is proven to be *sound* and *complete* with respect to minimal models: the property of *soundness* guarantees that every model derived from the application of the rules is actually consistent with the input traces, meaning no false models are produced; the property of *completeness* guarantees that no minimal model is omitted, meaning all minimal solutions are found. This means the rules derive all and only the minimal models compatible with the traces (Theorems 1 and 2 in [3]).

2.3 PAL

PAL (Plan-Act-Learn) [2] addresses a structurally different problem from the previous ones: the agent perceives the environment through continuous observations $x \in \mathbb{R}^n$ and must build from scratch both a discrete representation of the environment (extensional model) and a perception function that links continuous observations to abstract symbolic states, $\rho : Q \times S \rightarrow \mathbb{R}^+$, where $Q \subseteq \mathbb{R}^n$ is the continuous observation space and S is the finite set of abstract discrete states built by the agent during exploration. For each abstract state $s \in S$ and each observation $\mathbf{x} \in Q$, the value $\rho(\mathbf{x}, s) = p(\mathbf{x} \mid s)$ provides the likelihood of having perceived $\mathbf{x}$ while being in s . Every single observation of the environment is mathematically modeled as a vector of real-valued continuous variables $\mathbf{x} = \langle x_1, x_2, \dots, x_n \rangle$.

The extensional model $M = (S, A, \gamma)$ is a deterministic finite state machine: S is the set of discovered abstract states, A is the finite set of actions, and $\gamma : S \times A \rightarrow S$ is the deterministic transition function governing the system’s evolution. PAL builds this model online, during exploration: if the current observation does not correspond to

any known state (i.e., $\rho(x, s) < t$ for all $s \in S$), a new state is created. The perception function ρ is modeled as a product of univariate normal distributions and is updated with each new observation.

To guide exploration towards useful states, PAL uses a “draft” PDDL domain (not necessarily correct) as an exploratory model: when the extensional model does not contain a plan towards the goal, PAL invokes a PDDL planner on the draft domain and updates the draft domain every time an action fails in the real environment. The final output of PAL is not a set of PDDL operators with preconditions and effects, but the learned extensional model: the transition function γ between abstract states. The conversion of this extensional model into an intensional PDDL schema is a separate problem, not directly addressed by PAL.

2.4 Positioning of our framework

The proposed framework is positioned relative to the three reference works along four dimensions: learning modality, type of observations, mechanism for partial observability, and type of output.

Learning modality. Like OLAM [1], the system learns online during interaction with the environment, updating precondition and effect estimates with each newly observed transition. Both systems explicitly distinguish successful executions from failed ones, conditioning the model update only on successes. The difference lies in the data collection strategy: OLAM, at each iteration, actively builds a plan towards a state from which the execution of a specific action can provide new information. The system presented in this project instead adopts a simpler strategy: an execution phase of pre-constructed manual plans, followed by random exploration. The advantage is implementational simplicity; the disadvantage is that systematic coverage of all informative cases is not guaranteed.

Type of observations. Unlike PAL [2], states are already symbolic: the learner receives sets of discrete logical predicates, not continuous vectors. The `mask_prob` parameter models sensory imperfection at the symbolic level: with probability `mask_prob`, each predicate of the current state is independently removed from the observation before being delivered to the learner. For example, if the real state is $\{\text{at}(\text{robot}, 1, 1), \text{handempty}, \text{at}(\text{box}, 2, 2)\}$ and `mask_prob` = 0.3, the learner might receive only $\{\text{at}(\text{robot}, 1, 1), \text{at}(\text{box}, 2, 2)\}$ having lost `handempty`. The resulting problem is that the learner cannot distinguish “the predicate is false in the real state” from “the predicate is true but was obscured”: this is exactly the same ambiguity that OffLAMPT addresses with its logical completion rules, but here it is resolved with a statistical heuristic.

Mechanism for partial observability. The inertia completion implemented in this project is directly inspired by rule (8) of OffLAMPT [3]: if a literal l is present in state s before an action op and l is not among the potential effects of op , then l is propagated to the subsequent state s' . The fundamental difference is that OffLAMPT operates offline on a set of traces and applies completion rules to convergence with formally proven guarantees of soundness and completeness, while the inertia completion implemented here is applied heuristically and online, without the same theoretical guarantees. Furthermore, OffLAMPT also handles traces with partially observed actions (action completely or partially unknown), a case not addressed in this project.

Type of output. As in OLAM, our framework returns operators in a **lifted** format, meaning expressions where the constants of the observed objects have been abstracted and replaced with logical parameters. Unlike OffLAMPT, which selects minimal models via a precise order relation—a model $\mathcal{M}_1$ is smaller than $\mathcal{M}_2$ if the transitions enabled by $\mathcal{M}_1$ are a subset of those of $\mathcal{M}_2$ and each transition produces fewer state changes in $\mathcal{M}_1$ —this system uses statistical thresholds to filter preconditions and effects. In OffLAMPT the selection criterion is a logical property with provable guarantees; here it is a heuristic choice dependent on two hyperparameters.

In summary, the framework acts as an online action model learning system (like OLAM), operating on already discrete symbolic states (unlike PAL), and adopts a statistical and incremental version of the inertia principle (borrowed from OffLAMPT) to handle partial observability. Compared to each of the three reference systems, the approach sacrifices formal theoretical guarantees in favor of greater implementational simplicity and direct applicability in stochastic environments with actuator noise.

3 Agent Characteristics

Having defined in Section 2 the theoretical coordinates of the problem—online learning, partial observability, symbolic states—this section describes how these characteristics translate into concrete system design choices: the structure of the environment, the noise model, the data collection strategy, and the shape of the output.

Our agent operates in a discrete two-dimensional environment, but with partially non-deterministic dynamics. The `GridWorldStochastic` class models a rectangular world in which the robot can move, pick up an object, and drop it, but each of these actions can be disrupted by noise phenomena. The main characteristics of the system are therefore as follows.

3.1 Environment and symbolic representation

The state of the world is represented as a set of symbolic predicates. This translation is performed by the simulator whenever an observation is needed: the `state_predicates()` method of the `GridWorldStochastic` class reads the concrete position of the robot, the position of the object, and the gripping state, and builds the corresponding logical description. For example, if the robot is at position $(2, 3)$ and the object is in the same cell but is not being held, the state will contain $\{at(robot, (2, 3)), at(box, (2, 3)), handempty\}$; if instead the robot is holding the object, the state will contain $\{at(robot, (2, 3)), at(box, held), holding(robot, box)\}$ and will not contain `handempty`. This representation choice is important because it makes state transitions directly compatible with the structure of PDDL operators: the positive effects of an action are exactly the predicates that appear in s' but not in s , and the negative effects are those that appear in s but not in s' .

The environment is small but rich enough to require the discovery of non-trivial relationships. The pick action, for example, presupposes the co-localization of the robot and the object and simultaneously modifies multiple predicates: the object moves

from a physical position to the `held` state, the `holding` predicate appears, `handempty` disappears, and the predicate placing the object in the previous cell is removed. Similarly, the drop action must reverse these transformations.

3.2 Partial observability

A central feature of the system is partial observability. After each action execution, the agent does not observe the complete state of the world but a censored version: each predicate that would be present in the complete description is independently eliminated from the observation with probability `mask_prob`. Concretely, if the real state after an action contains five predicates and `mask_prob = 0.4`, each of the five has a 40% probability of not appearing in the observation delivered to the learner; on average three survive, but which three varies with each transition. The learner thus receives a set of predicates that is always a subset of the real state, without being able to distinguish obscured predicates from genuinely absent ones.

From a learning perspective, the partiality of the observation greatly complicates the problem. If a predicate does not appear in the subsequent observed state, it is not immediately possible to establish whether it was removed as an effect of the action or whether it simply escaped the sensors. To solve this problem, the project introduces an *inertia completion* step, which is a prudent propagation of facts that most likely remained unchanged.

3.3 Environment noise (non-determinism)

The system models three different sources of noise on the actuators:

- `slip_prob`: probability that the robot does not execute the desired movement, but slips in an alternative direction or remains stationary;
- `pick_fail_prob`: probability that the pick action fails even if the object is located in the same cell as the robot;
- `drop_misplace_prob`: probability that the dropping of the object occurs in a wrong position compared to the robot's position.

This threefold modeling is particularly effective because it distinguishes different types of uncertainty: locomotion noise, manipulation noise, and release noise. The agent does not simply observe correct transitions but must also learn to handle anomalous executions. In the code, failed transitions are still recorded in the global history, but are excluded from the direct estimation of effects, so as to prevent stochastic failures from contaminating the identification of the nominal action model.

3.4 Hybrid learning: random exploration and manual plans

The agent learns by combining two sources of experience. The first is a phase guided by *manual plans*, i.e., sequences of actions built by hand and repeated multiple times in different initial configurations. This phase serves to ensure coverage of actions of interest, specifically `pick` and `drop`. `pick` and `drop` have specific preconditions that random exploration rarely satisfies: `pick` requires the robot and object to be in the

same cell, `drop` requires the robot to already be holding the object. In a 4x4 grid with random positions, the probability that `pick`'s preconditions are met by chance is 1/16 per step. With only random exploration, these operators would accumulate an insufficient number of examples for statistical estimation within a reasonable number of episodes.

The second source is random exploration over a set of candidates, where the system dynamically chooses among movement, picking, and dropping when their respective operational preconditions are met. In this way, manual plans provide informative examples, and random exploration adds variety, edge cases, and natural noise.

3.5 Final output of the agent

The agent's output is a set of operators in a readable *pseudo-PDDL* format produced by the `print_parametric_pddl` function. The term "pseudo" indicates that the structure follows PDDL syntax, but negative effects are labeled as `del-effects` instead of the standard `(not (...))` syntax. An example for the `pick` operator, inferred after convergence in the absence of noise, is the following:

```
(:action pick

:parameters (?robot ?obj ?x1 ?y1)

:precondition (and (at ?robot ?x1 ?y1)
                  (at ?obj ?x1 ?y1)
                  (handempty ?robot))

:add-effects  (holding ?robot ?obj)
              (at ?obj held)

:del-effects  (at ?obj ?x1 ?y1)
              (handempty ?robot))
```

Each predicate is accompanied by the empirical support frequency with which it was observed in the corresponding successful transitions.

The features described in this section correspond to specific implementation components. Section 4 analyzes each of them in detail: the simulator (4.2), the learner and inertia completion (4.3, 4.4), the generation of experiences (4.6), and the parametric inference function (4.5).

4 Agent Implementation

This section analyzes the structure of the project from a code perspective. The implementation is organized as a pipeline consisting of a simulator, learner, parametric abstraction, and experimental blocks.

4.1 Fundamental mathematical notation

To understand the data collection logic and statistical inference implemented in the code, it is useful to define the notation used to describe transitions:

- s and s' : represent respectively the state of the world before and after the execution of an action, formally understood as sets of logical predicates.
- p : a single logical predicate (or atom) describing a property of the world (e.g., $p \in s$).
- a_s : a *successful* execution of a specific action. The subscript emphasizes that the inference is conditioned exclusively on successes, discarding stochastic failures.
- $N(a_s)$: the sample size, i.e., the total number of successful executions recorded for a specific action.
- $(s' \setminus s)$: the set of *additive effects* (or positive), understood as the novelties introduced by the action in the new state compared to the previous one.
- $(s \setminus s')$: the set of *subtractive effects* (or negative), understood as the predicates that did not survive the transition.
- $T_s(a)$: the set of all successful transitions for action a , represented as a set of pairs (s, s') .

4.2 Environment Simulator, `GridWorldStochastic`

The `GridWorldStochastic` class forms the backend of the entire project. The constructor initializes the grid dimensions ($W \times H$), the robot's position, the object map, and the noise parameters. From a logical point of view, the four most important functions are:

The first is `reset`, which sets a new initial configuration for the robot and object. The configuration is randomly extracted from a set of predefined variants: the robot and object start from different positions on the grid, chosen to guarantee variety in learning experiences and coverage of different spatial arrangements.

The second is `state_predicates`, which translates the concrete state into a set of symbolic predicates, effectively generating the formal states s and s' analyzed by the *Learner*.

The third and fourth are `_apply_pick` and `_apply_drop`, which implement the grasping and releasing dynamics, distinguishing between *applicability* and *success*. An action is *applicable* if its logical preconditions are met in the current state: for `pick`, the robot and object must be in the same cell; for `drop`, the robot must already be holding the object. An applicable action can, however, *fail* due to stochastic noise: `pick_fail_prob` is the probability that grasping does not succeed even if preconditions are met, and `drop_misplace_prob` is the probability that releasing occurs in a different cell from the robot's. This separation is essential for the learner: an execution can be applicable but not successful, and the code explicitly represents this via the `failed` flag.

The `step` function coordinates the entire environment dynamics in four phases: (1) determines which action to execute and verifies that its preconditions are satisfied in

the current state; (2) applies the appropriate noise for the action type—random slip for `move`, failure probability for `pick`, dislocation for `drop`; (3) updates the simulator’s internal state if the action succeeds; (4) sets the `failed_this_step` flag to `True` if the action failed for any reason, whether due to unmet preconditions or stochastic noise. The flag is then delivered to the learner along with the transition (s, s') , allowing it to separate useful data—successful transitions—from noisy ones.

4.3 Transition memory and statistical estimation, `MiniActionModelLearner`

The symbolic learning component is implemented in the `MiniActionModelLearner` class. This class does not attempt to learn a policy, but rather an action model. Its main attributes are:

- `state_history`, which stores the entire history of observed transitions;
- `pre_examples`, which keeps the states observed before the successful executions of each operator;
- `eff_pos_counts` and `eff_neg_counts`, which count respectively the facts that appeared and disappeared between initial and final states;
- `sa_counts` and `fail_counts`, which keep track of successes and failures per operator.

The `observe_transition` method is the heart of data collection. It saves every observed transition and, if the action has not failed, updates the counts for preconditions and effects.

The `compute_preconditions` method estimates the preconditions by thresholding the co-occurrence frequency: a predicate p is kept as a precondition of operator a if its co-occurrence frequency with successful executions exceeds the threshold τ_{pre} . Formally, the empirical probability that a predicate p is a precondition is:

$$P_{\text{pre}}(p, a) = \frac{|\{s \in \text{pre_ex}(a) : p \in s\}|}{N(a_s)}$$

where $N(a_s)$ is the total number of successful executions of a . A predicate p is definitively maintained as a logical precondition of the operator if and only if this probability reaches the threshold τ_{pre} :

$$p \in \text{Pre}(a) \iff P_{\text{pre}}(p, a) \geq \tau_{\text{pre}}$$

A value $P_{\text{pre}}(p, a) = 0.9$ with $\tau_{\text{pre}} = 0.7$ indicates that p was present in 90% of the initial states observed before successes of a : since $0.9 \geq 0.7$, the predicate is kept.

This heuristic is justified by the assumption that true domain preconditions appear almost always in the initial state of successful executions; deviations below the threshold τ_{pre} are attributed to sensory masking (`mask_prob` > 0). The limit of the approach is that the threshold τ_{pre} introduces a trade-off. With thresholds too low, contextually true but not causally necessary predicates—for instance, a static predicate always true in the environment—are included as spurious preconditions. With thresholds

too high in the presence of high sensory masking, predicates that are true preconditions but get hidden in a significant portion of observations risk being erroneously discarded.

The `compute_effect_probabilities` method calculates the empirical probability of each positive and negative effect relative to the number of operator successes. The empirical probability of a positive effect p is:

$$P_{\text{add}}(p, a) = \frac{|\{(s, s') \in T_s(a) : p \in (s' \setminus s)\}|}{N(a_s)}$$

and of a negative effect:

$$P_{\text{del}}(p, a) = \frac{|\{(s, s') \in T_s(a) : p \in (s \setminus s')\}|}{N(a_s)}$$

A positive (or negative) effect becomes part of the operator's logical model if and only if its empirical probability exceeds the confidence threshold τ_{eff} :

$$p \in \text{ADD}(a) \iff P_{\text{add}}(p, a) \geq \tau_{\text{eff}}$$

$$p \in \text{DEL}(a) \iff P_{\text{del}}(p, a) \geq \tau_{\text{eff}}$$

where $T_s(a)$ is the set of successful transitions recorded for a . A positive effect p is included in the operator if its empirical probability $P_{\text{add}}(p, a)$ —the fraction of successful transitions in which p appeared as a new fact after executing a —exceeds the `eff_thresh` threshold. Symmetrically, a negative effect is included if $P_{\text{del}}(p, a)$ exceeds the same threshold. A threshold of 0.1 means an effect is included if observed in at least 10% of successful executions; higher values reduce false positives but risk losing rare or partially masked effects.

4.4 Inertia Completion, `apply_inertia`

One of the most interesting steps is the `apply_inertia` method. The underlying logic is as follows: if a fact disappears from the observed state, but is not supported as a true negative effect of the action, then it is plausible that it has not really disappeared and its absence depends solely on masking. Symmetrically, if a fact appears in the subsequent state without being a plausible positive effect, the method propagates it backward into the previous state.

Formally, given a partially observed state s'_{obs} produced by an action a applied to s , inertia completion produces an extended state s'_{ext} defined as:

$$s'_{\text{ext}} = s'_{\text{obs}} \cup \{p \in s \mid \text{sig}(p) \notin \text{sig}(\text{DEL}_{\text{est}}(a)) \wedge p \notin s'_{\text{obs}}\} \quad (\text{forward propagation})$$

$$s_{\text{ext}} = s \cup \{p \in s'_{\text{obs}} \mid \text{sig}(p) \notin \text{sig}(\text{ADD}_{\text{est}}(a)) \wedge p \notin s\} \quad (\text{backward propagation})$$

where $\text{sig}(p)$ is the signature of the predicate (name of the predicate and arity, without concrete constants), $\text{DEL}_{\text{est}}(a)$ and $\text{ADD}_{\text{est}}(a)$ are the negative and positive effects currently estimated for a . Forward propagation keeps persistent predicates in the

new state; backward propagation adds them to the previous state if they appear in the new one without being expected effects. The comparison is done via the internal `get_sig` function, which builds a simplified signature of the atom containing only the predicate name and the argument name that identifies the *relation type*, without concrete coordinate values. For example, `sig(at(robot,1,1)) = sig(at(robot,2,2)) = "at:robot"`: the two signatures are identical even if the atoms are different. If an atom's signature matches the signature of an expected action effect—meaning the action is known to modify that relationship—inertia is not applied, even if the concrete predicate does not explicitly appear in the list of certain effects. This avoids the problematic case where the robot moved from (1,1) to (2,2): `at(robot,1,1)` should not be propagated to the next state, because the "robot position" property changed even though `at(robot,1,1)` does not appear directly among certain negative effects.

Conceptually, this module is fundamental because it transforms a partial observation problem into a denser set of transitions, thus more useful for subsequent symbolic abstraction. Without this step, many preconditions and effects would be underestimated or lost.

4.5 Parametric inference of operators

The `infer_parametric_operators_from_history` function performs the most delicate step of the whole project: transforming a collection of concrete transitions into abstract operators. Its functioning can be summarized in four stages.

In the first stage, it groups transitions by action type, using only the base name of the operator. Thus, all instances of `move`, regardless of direction, are aggregated together. In the second stage, it calculates the set of observed preconditions, positive effects, and negative effects for each successful transition.

In the third stage, *lifting* comes into play. Lifting operates by building a substitution map that assigns a symbolic variable to each concrete constant present in the transition. The assignment follows recognizable roles by pattern matching on predicate names: the constant appearing as the first argument of `at(·,·)` in a context associated with the robot is mapped to `?robot`; the constant appearing in `holding(·,·)` is mapped to `?obj`; the (x,y) coordinates of the robot's position in the initial state are mapped to $(?x1, ?y1)$; those in the final state to $(?new_rx1, ?new_ry1)$. The result is that the concrete transition "the robot was at (2,3), executed `move`, is now at (2,4)" becomes the lifted instance `at(?robot, ?x1, ?y1) → at(?robot, ?new_rx1, ?new_ry1)`.

At the end of lifting, in the fourth stage, for each lifted atom its frequency of appearance among all transitions in the history is counted: this normalized count is its *empirical support*. Only atoms whose support exceeds the `precondition_prob_threshold` are kept as preconditions; similarly for effects with the `effect_prob_threshold`. For example, if a lifted predicate appears as a precondition in 45 out of 50 transitions, its support is 0.90; with a threshold of 0.70 it is maintained.

The final result is a dictionary `ops_info` which, for each operator, contains a list of parameters, preconditions, add effects, and delete effects. The `print_parametric_pddl` function uses this structure to print a pseudo-PDDL version readable by the user.

4.6 Generation of experiences

The `run_experiment` function orchestrates the entire experimental cycle. It initializes the environment with a 4×4 grid, sets up the learner, and defines a set of initial variants of the world. Subsequently, it executes two phases:

1. an execution phase of manual plans, repeated multiple times to increase action coverage and obtain significant examples of `pick` and `drop`;
2. a random exploration phase with hundreds of episodes, in which action candidates are chosen dynamically based on the context.

This design is important because it solves a classic problem of action model learning: some actions are rare, and if only random exploration is used, they may not accumulate sufficient support. The presence of the `manual_plan_executions` parameter allows deliberately increasing the amount of informative examples.

At the end of the experiment, the code calculates the effect probabilities, applies inertia to the history, and invokes parametric lifting. The returned value is thus already a symbolic model ready to be inspected.

4.7 Validation of inferred operators: two distinct levels

The project adopts two validation functions with different purposes and rigor, used in different phases of the pipeline.

4.7.1 `check_pddl_inference`: structural completeness check

This function, used in iterative scenario experiments, simply verifies that the inferred operator is *structurally complete*: it checks that the four fundamental fields of the operator (`params`, `pre`, `add`, `del`) are all non-empty. It does not evaluate the semantic coherence of the predicates nor the correctness of spatial relations; it merely ensures that the learner has produced something for each component. This is deliberately a weak criterion, suitable for monitoring convergence during iterations.

4.7.2 `is_strictly_valid`: UPF-based semantic validator

This function, used exclusively in the heatmap generation phase, is the true independent judge of the project. It converts the inferred operator into an action of the *Unified Planning Framework* (UPF) and applies semantic checks specific to each action. The semantic validation of `move` verifies three conditions on the coordinates:

1. The `at` predicate of the robot in the precondition and that in the delete effect must refer to the same coordinates (the robot starts from the position that is removed):

$$\text{coord}(\text{PRE}_{\text{robot-at}}) = \text{coord}(\text{DEL}_{\text{robot-at}})$$

2. The coordinates in the add effect must be different from those at the start (the robot actually moves):

$$\text{coord}(\text{ADD}_{\text{robot-at}}) \neq \text{coord}(\text{DEL}_{\text{robot-at}})$$

3. There must exist exactly one `at` predicate of the robot in each of the three sets (precondition, add-effect, del-effect).

where `coord(p)` extracts the (x, y) pair from the `at(robot, x, y)` predicate. Similar conditions are verified for `pick` (co-localization in preconditions) and `drop` (object's position in the add-effect equal to the robot's in the precondition).

Each comparison is recorded in the `upf_validation.log` file to ensure traceability. It is important to keep this distinction in mind when reading the results: the narrative scenarios use `check_pddl_inference` (weak criterion), while the heatmaps use `is_strictly_valid` (strong criterion). The two types of results are therefore not directly comparable.

4.8 Iterative experiments and final dashboard

The project also introduces `run_iterative_experiment_for_scenarios`, a function that runs experiments in multiple scenarios and progressively increases the number of manual plans until the three actions `move`, `pick`, and `drop` are all satisfactorily inferred according to `check_pddl_inference`. The actual call in the code uses `initial_executions=20` and `increment_by=10`: the starting point is therefore 20 manual executions, and each iteration adds 10 further executions up to a maximum of 10 overall iterations. This part is significant because it transforms the project into a systematic testbed.

There are four main scenarios: ideal baseline, high environmental noise, low visibility, and a fourth scenario, in which only 50 episodes are executed.

For the heatmap generation, common thresholds are fixed for all scenarios regardless of narrative parameters: `FISSO_PRE_THRESH = 0.4` and `FISSO_EFF_THRESH = 0.1`, with `MANUAL_PLAN_EXECS = 80` and `NUM_EXECUTIONS = 4` repetitions per cell. These constants differ from the scenario-specific thresholds used in the iterative experiments (where `pre_thresh` varies between 0.6 and 0.8 depending on the scenario), which makes the two types of experiments not directly comparable.

The final quantitative phase of the project is entrusted to the `build_mega_grids_and_plot` function, which builds success rate matrices for `move`, `pick`, and `drop` on a 6×6 grid of actuator noise and sensory masking levels (both from 0.0 to 0.5 in steps of 0.1), repeating each cell `NUM_EXECUTIONS = 4` times and using fixed thresholds `FISSO_PRE_THRESH = 0.4` and `FISSO_EFF_THRESH = 0.1` with `MANUAL_PLAN_EXECS = 80` manual executions. The validation of each operator is handled by `is_strictly_valid`.

A relevant methodological element is the adaptive convergence criterion that sets the number of episodes. The function progressively increments the number of training episodes in steps of 100 starting from 1000, and stops when the maximum improvement across all 21 graphs (7 scenarios $\times$ 3 actions) falls below the 5% threshold for 3 consecutive iterations. Given the randomness of the noise, convergence is reached with a number of episodes that varies depending on the execution. The episode interval that allows convergence is generally between 1600 and 2300. In the reported execution, convergence was reached at **1600 episodes**: this value empirically quantifies how much training is sufficient for the system to produce stable evaluations, avoiding both underestimation (too few episodes, noisy results) and computational waste (excess episodes without accuracy gain). The produced heatmaps represent

the clearest quantitative part of the entire project, as they show at a glance where the agent succeeds or fails to reconstruct semantically correct operators.

One element to highlight is the presence of actual experimental diagnostic instrumentation. The

`run_iterative_experiment_for_scenarios` routine does not merely stop when `move`, `pick`, and `drop` are inferred, but prints for each iteration a summary of the total number of executions, successes, and failures for each action. It also saves this information in the `detailed_inference_results` structure, which is then displayed in tabular form per scenario. Added to this are three qualitative analysis tools: the printing of *unexpected outcomes*, which lists anomalous transitions or stochastic failures; the `infer_domain_config_from_observations` function, which derives the number of objects and grid size from history and observed predicates; and the `debug_inertia_to_file` function, which exports a step-by-step explanation of the inertia completion to a file. Consequently, the project provides not just a final model, but also a set of printouts and artifacts useful for interpreting how the model is obtained.

4.9 Agent Pseudocode

To clarify the project's operation, it is useful to summarize the pipeline in algorithmic form. The first pseudocode summarizes the overall flow of `run_experiment`: environment initialization, transition collection from manual plans and random exploration, effect estimation, inertia application, and final production of parametric operators. The second pseudocode instead describes the learner's behavior when it observes a new transition.

Algorithm 1 Main domain inference pipeline

Require: Noise parameters, inference thresholds, number of episodes, number of manual plan executions

```
1: Initialize environment GridWorldStochastic
2: Initialize learner MiniActionModelLearner
3: Define initial variants of the world and set of manual plans
4: for all manual plan  $\pi$  do
5:   for  $i \leftarrow 1$  to number of executions of  $\pi$  do
6:     Extract a random initial configuration and perform reset
7:     for all action  $a \in \pi$  do
8:       Partially observe the current state  $s$ 
9:       if  $a$  is applicable in the current state then
10:        Execute step(a) and obtain next state  $s'$  and failure flag
11:        Partially mask  $s'$ 
12:        Update the learner with  $(s, a, s', failed)$ 
13:      end if
14:    end for
15:  end for
16: end for
17: for  $ep \leftarrow 1$  to number of random episodes do
18:   if  $ep$  is a multiple of 25 then
19:     Reinitialize the world by randomly extracting a new initial configuration
    from the list of predefined variants
20:   end if
21:   Partially observe the current state
22:   Construct the set of candidate actions: move is always available in the four
    cardinal directions; pick is a candidate if the robot and object are in the same cell;
    drop is a candidate if the robot is currently holding the object
23:   Randomly extract a candidate action
24:   if the chosen action is valid then
25:     Execute the action, observe the new masked state, and update the learner
26:   end if
27: end for
28: Calculate the empirical probabilities of the effects
29: Apply inertia completion to the observed history
30: Infer parameterized operators from completed history and statistical thresholds
31: return learner, inferred operators, effect probabilities
```

Algorithm 2 Learner update on an observed transition

Require: Observed state s , action a , next observed state s' , flag failed

```
1: Save  $(s, a, s', failed)$  in the global history
2: if failed is true then
3:   Increment the failure counter for the operator of  $a$ 
4:   return
5: end if
6: Increment the success count of the operator of  $a$ 
7: Add  $s$  to the precondition examples of the operator
8: for every atom  $p$  such that  $p \in s' \setminus s$  do ▷ new facts: positive effects
9:   Increment the positive effects count for  $a$ 
10: end for
11: for every atom  $p$  such that  $p \in s \setminus s'$  do ▷ disappeared facts: negative effects
12:   Increment the negative effects count for  $a$ 
13: end for
```

These two algorithms clearly highlight the project’s philosophy: the agent only learns from successful executions, but nevertheless keeps track of failures to characterize the difficulty of the environment. Moreover, PDDL inference does not happen directly on raw data, but rather on a history “cleaned” by the inertia step and then generalized via symbolic lifting.

5 Results Obtained

The results of the project can be read on three levels: the first concerns the narrative scenarios of the project, the second concerns the heatmap dashboard that summarizes the agent’s robustness as noise varies, and the third concerns the diagnostic printouts per iteration. These printouts explicitly report, for each action, how many total executions were observed, how many were successful, and how many were classified as failures.

5.1 Results of the main scenarios

Some very useful data emerge from the scenarios. In the *Ideal Baseline* case, the system correctly infers all operators right at the first iteration, with 20 manual executions. This outcome is consistent with theoretical expectations: in the absence of actuator noise ($slip_prob = pick_fail_prob = drop_misplace_prob = 0$) and with full observability ($mask_prob = 0$), every observed transition is an unbiased sample of the action’s nominal behavior. Under these conditions, 20 manual executions are sufficient to bring the empirical frequency of every true precondition above the threshold τ_{pre} and that of every true effect above τ_{eff} .

This intermediate view is useful for at least three reasons. First, it allows distinguishing scenarios where the model converges rapidly from those where it is necessary to accumulate many more observations. Second, it makes clear that the bottleneck is not always the same: in some cases the problematic action is `move`, in others it is

mainly `pick` and `drop`. Third, it highlights the role of noise, because a high number of failures or stochastic deviations directly reflects in a slowdown of inference.

Scenario	Move	Pick/Drop	Manual executions needed
Ideal Baseline	yes	yes	20
High environmental noise	yes	yes	30
Low visibility	yes	yes	70
Scenario 4 (Low Exploration)	yes	yes	20

Table 1: Summary of the main results extracted from the project outputs with seed=42. Validation is based on `check_pddl_inference` (structural completeness).

5.2 Qualitative comment on inferred operators

The operators produced by the system are semantically correct in configurations with low or moderate noise. The most frequent deviations occur in two specific patterns: (1) for `pick`, when $\text{mask_prob} \geq 0.3$, the `handempty` predicate is omitted from the preconditions because its co-occurrence frequency drops below τ_{pre} — the robot almost always holds it at the beginning of a successful execution of `pick`, but masking makes it invisible in a sufficient quota of transitions to make it seem optional; (2) for `drop`, when $\text{drop_misplace_prob} > 0$, the coordinates of the object in the additive effect do not match those of the robot in the precondition in a growing portion of successful transitions (because the release happens in an adjacent cell), which causes the semantic check of `is_strictly_valid` to fail even when the operator structure is otherwise correct.

These results show that parametric lifting works correctly at least on the fundamental roles of the domain. The system doesn't just memorize concrete constants, but learns a reusable abstract structure. The probabilities associated with the predicates are also informative: when an effect appears with a probability of less than 1, the report allows understanding that the action was observed in the presence of noise or partial concealment of facts.

5.3 Final graph analysis

Figure 1 shows the new final dashboard generated by the project. Unlike the previous version, here a single summary heatmap is not shown, but a grid of separate heatmaps: the rows correspond to different scenario families, while the three columns represent respectively the `move`, `pick`, and `drop` operators. In each sub-graph, the vertical axis describes the action noise level and the horizontal axis the sensory masking level; the color and percentage in the cell indicate how many executions produced an operator deemed semantically valid by the `is_strictly_valid` validator.

The dashboard shown relates to a specific execution where experimental convergence was reached after **1600 episodes**. This value should be interpreted as the point where, for this run, the system accumulated enough observations to produce

a stable map of successes and failures. Due to the stochastic component of the simulator, other executions might reach convergence with a slightly different number of episodes; however, broadly speaking, the qualitative behavior remains the same in other runs: the green zones remain concentrated in configurations with low or moderate noise, while red cells appear mostly when actuator noise and sensory masking increase together.

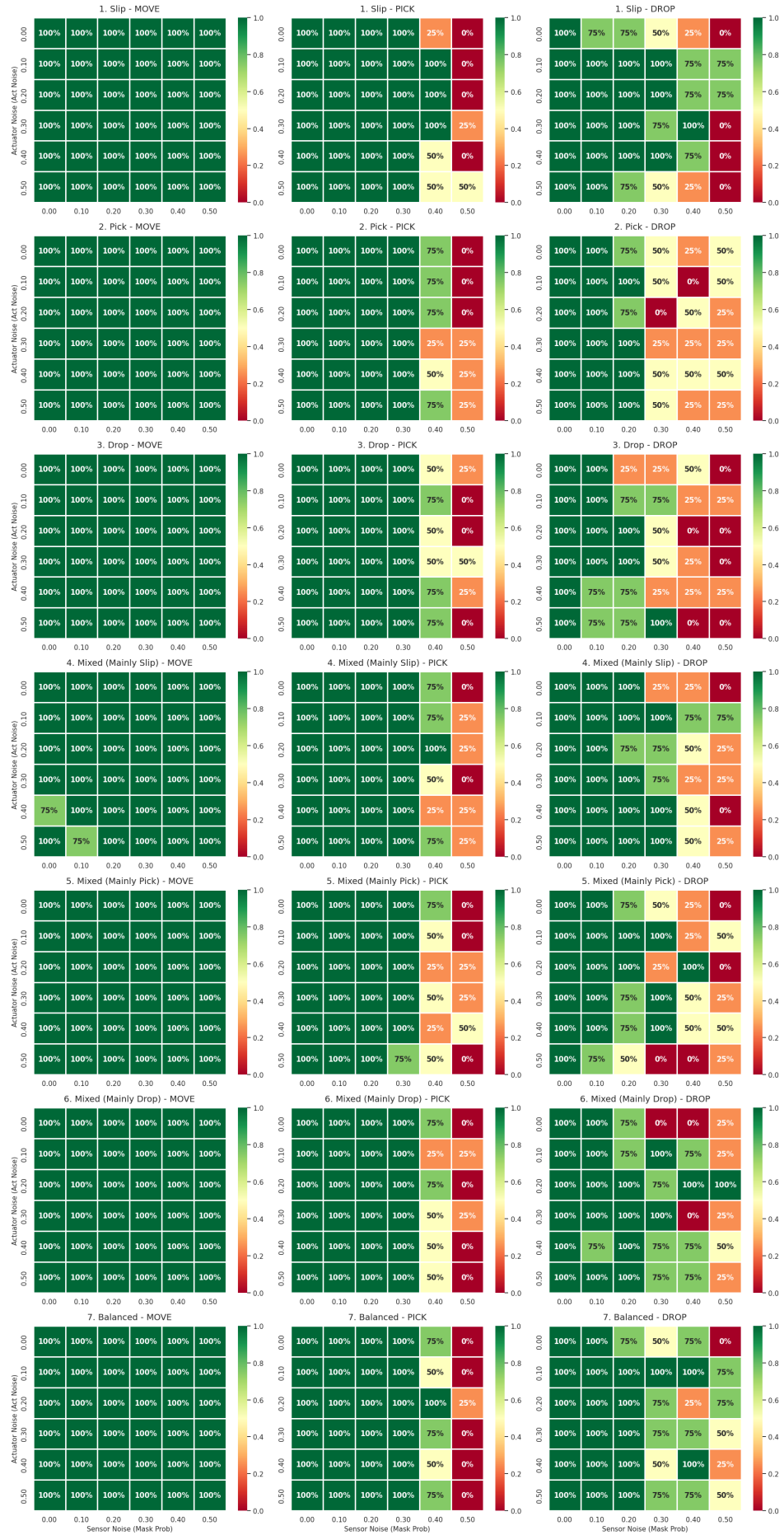


Figure 1: Experimental dashboard of the converging execution at 1600 episodes. Each sub-heatmap shows the correct inference rate of one of the operators `move`, `pick`, and `drop` as actuator noise and sensory masking vary, according to the semantic validator `is_strictly_valid`.

The first visible pattern is the greater robustness of `move`. In most scenarios, the heatmaps related to `move` remain almost completely green, with percentages almost always at 100%. This result is consistent with the action’s structure: `move` primarily depends on the robot’s position and does not require correctly observing complex relationships between multiple objects. Even when masking increases, the agent often manages to recover sufficient information via inertia completion and statistical accumulation of transitions.

The second pattern concerns `pick`, which turns out to be more sensitive to noise increase. In the related heatmaps, more orange and red cells appear, especially on the right side, where sensory masking is highest. This trend is expected because `pick` requires simultaneously recognizing the co-localization between robot and object, the appearance of `holding`, the disappearance of the object’s physical position, and the loss of `handempty`. If any of these pieces of evidence is hidden or altered, the inferred operator may not pass the stricter semantic check.

The third pattern concerns `drop`. The heatmaps show behavior that is often more unstable than `move`: the action works well in favorable conditions but degrades when release noise and sensory masking become high. The reason is that `drop` combines a strong symbolic precondition, namely the fact that the object is already held by the robot, with a spatial effect that must be observed with sufficient precision. When the object is dropped in an unexpected position or when the final predicates are masked, operator validation becomes more difficult.

One last important aspect is that the boundaries between green and red cells are not identical in all scenarios. This confirms that the effect of noise is not simply linear: a certain level of masking can be tolerated when actuator noise is low, but becomes much more damaging when actions also fail or produce stochastic outcomes.

5.4 Overall Evaluation

Overall, the results are positive. The project demonstrates that even with a relatively simple learner, it is possible to reconstruct interpretable PDDL-like operators starting from raw observations. The main strength of the system is the robustness of `move`: in all seven tested noise scenarios, the action is correctly inferred in 100% of executions even with masking levels up to 0.4. This result is attributable to the structural simplicity of the operator (precondition on a single position, two symmetric effects) and the natural abundance of examples during the exploration phase. For `pick` and `drop`, quality degrades more rapidly, but the system produces semantically correct operators in a large portion of the tested noise space ($\text{mask_prob} \leq 0.2$ for most scenarios). In terms of interpretability, every phase of the pipeline is traceable: counts

per iteration, inertia logs, and the printing of *unexpected outcomes* allow diagnosing exactly where the inference process diverges. Furthermore, the combination of probabilistic thresholds, failure filtering, and inertia makes the system surprisingly robust in non-trivial scenarios.

The limitations emerge primarily when the domain requires more subtle dependencies or when observability drops significantly. In these cases, the method may require many more guided executions and may degrade more rapidly on manipulation actions compared to movement ones. This is nonetheless an interesting result because it precisely identifies which domain components are easier to learn and which instead require more sophisticated exploration strategies or richer probabilistic models.

6 Conclusions

The experimental results allow us to answer the central question of the project in a differentiated manner. On the structural completeness criterion (`check_pddl_inference`), the system is robust: all three operators are inferred in all scenarios within a maximum of 70 manual executions. On the semantic criterion (`is_strictly_valid`), which is the most rigorous and meaningful judge, robustness is high for `move`, moderate for `pick`, and variable for `drop`, with a clear dependence on the levels of actuator noise and sensory masking. This means the system produces structurally complete operators almost always, but semantically correct operators only under conditions where the available information is sufficient for lifting. The distinction between the two criteria is not a technical detail: it is the most honest measure of the real limits of the system.

The agent presented in the project convincingly achieves the project’s goal: to learn a PDDL-style symbolic domain starting from observed plans and executions. The pipeline starts from an explicit stochastic environment, collects transitions while filtering failures, partially completes observations via inertia, and finally produces readable parametric operators. The code analysis highlights some design choices: a clear separation between environment and learner, use of thresholds for preconditions and effects, explicit noise handling, scenario-based experiments, and an interpretable final representation. At the same time, the experimental results show where the real difficulties lie: not so much in learning `move`, but in reconstructing actions like `pick` and `drop` when observations and dynamics become unreliable. As a future evolution, the project could be extended in at least three directions: using multiple objects of different types, introducing negative or conditional preconditions, and automatically validating the inferred operators on an external planner.

References

- [1] L. Lamanna, A. Saetti, L. Serafini, A. E. Gerevini, P. Traverso. *Online Learning of Action Models for PDDL Planning*. In Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI-21), pp. 4112–4118, 2021.

- [2] L. Lamanna, A. E. Gerevini, A. Saetti, L. Serafini, P. Traverso. *On-line Learning of Planning Domains from Sensor Data in PAL: Scaling up to Large State Spaces*. In Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI-21), pp. 11862–11869, 2021.
- [3] L. Lamanna, L. Serafini, A. Saetti, A. E. Gerevini, P. Traverso. *Lifted Action Models Learning from Partial Traces*. Artificial Intelligence, 339: 104256, 2025.